

卷

卷

Kubernetes **卷**为 中的容器提供了一种通过文件系统访问和共享数据的方式。存在不同类别的卷，你可以将其用于各种用途，例如：

- 基于 或 填充配置文件
- 为 Pod 提供一些临时的涂销空间
- 在同一个 Pod 中的两个不同容器之间共享文件系统
- 在两个不同的 Pod 之间共享文件系统（即使这些 Pod 运行在不同的节点上）
- 持久化存储数据，这样即使 Pod 重启或被替换，存储的数据仍然可用
- 基于容器所在 Pod 的详细信息，将配置信息传递给运行在容器中的应用（例如告诉：Pod 运行在哪个命名空间）
- 以只读权限访问另一个容器镜像中的数据

数据共享可以发生在容器内不同本地进程之间，或在不同容器之间，或在多个 Pod 之间。

为什么卷很重要 {#why-volumes-are-important}

- **数据持久性：** 容器中的文件在磁盘上是临时存放的，这给在容器中运行较重要的应用带来一些问题。当容器崩溃或被停止时，容器的状态不会被保存，因此在容器生命期内创建或修改的所有文件都将丢失。在崩溃之后，kubelet 会以干净的状态重启容器。
- **共享存储：** 当多个容器在一个 Pod 中运行并需要共享文件时，会出现另一个问题。那就是在所有容器之间设置和访问共享文件系统可能会很有难度。

Kubernetes 这一抽象概念能够解决这两个问题。

在你学习卷、持久卷（PersistentVolume）和持久卷申领（PersistentVolumeClaim）之前，你应该先了解，确保你理解 Kubernetes 如何使用 Pod 来运行容器。

卷是如何工作的 {#how-volumes-work}

Kubernetes 支持很多类型的卷。可以同时使用任意数目的卷类型。临时卷类型将生命周期关联到特定的 Pod，但持久卷可以比任意独立 Pod 的生命期长。当 Pod 不再存在时，Kubernetes 也会销毁临时卷；不过 Kubernetes 不会销毁持久卷。对于给定 Pod 中任何类型的卷，在容器重启期间数据都不会丢失。

卷的核心是一个目录，其中可能存有数据，Pod 中的容器可以访问该目录中的数据。所采用的特定的卷类型将决定该目录如何形成的、使用何种介质保存数据以及目录中存放的内容。

使用卷时，在 `.spec.volumes` 字段中设置为 Pod 提供的卷，并在 `.spec.containers[*].volumeMounts` 字段中声明卷在容器中的挂载位置。

当 Pod 被启动时，容器中的进程看到的文件系统视图是由它们的初始内容以及挂载在容器中的卷（如果定义了的话）所组成的。其中根文件系统同容器镜像的内容相吻合。任何在该文件系统下的写入操作，如果被允许的话，都会影响接下来容器中进程访问文件系统时所看到的内容。卷被挂载在镜像中的指定路径下。Pod 配置中的每个容器必须独立指定各个卷的挂载位置。

卷不能挂载到其他卷之上（不过存在一种使用 `subPath` 的相关机制），也不能与其他卷有硬链接。

卷类型 {#volume-types}

Kubernetes 支持下列类型的卷：

configMap

configMap 卷提供了向 Pod 注入配置数据的方法。ConfigMap 对象中存储的数据可以被 configMap 类型的卷引用，然后被 Pod 中运行的容器化应用使用。

引用 configMap 对象时，你可以在卷中通过它的名称来引用。你可以自定义 ConfigMap 中特定条目所要使用的路径。下面的配置显示了如何将名为 `log-config` 的 ConfigMap 挂载到名为 `configmap-pod` 的 Pod 中：

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
```

```

- name: test
  image: busybox:1.28
  command: ['sh', '-c', 'echo "The app is running!" && tail -f /dev/null']
  volumeMounts:
    - name: config-vol
      mountPath: /etc/config
volumes:
- name: config-vol
  configMap:
    name: log-config
    items:
      - key: log_level
        path: log_level.conf

```

log-config ConfigMap 以卷的形式挂载，并且存储在 log_level 条目中的所有内容都被挂载到 Pod 的 /etc/config/log_level.conf 路径下。请注意，这个路径来源于卷的 mountPath 和 log_level 键对应的 path。

- 你必须先创建 ConfigMap，才能使用它。
- ConfigMap 总是以 readOnly 的模式挂载。
- 某容器以 subPath 卷挂载方式使用 ConfigMap 时，若 ConfigMap 发生变化，此容器将无法接收更新。
- 文本数据挂载成文件时采用 UTF-8 字符编码。如果使用其他字符编码形式，可使用 binaryData 字段。

downwardAPI {#downwardapi}

downwardAPI 卷用于为应用提供数据。在这类卷中，所公开的数据以纯文本格式的只读文件形式存在。

容器以 subPath 卷挂载方式使用 downward API 时，在字段值更改时将不能接收到它的更新。

更多详细信息请参考通过文件将 Pod 信息呈现给容器。

emptyDir {#emptydir}

对于定义了 emptyDir 卷的 Pod，在 Pod 被指派到某节点时此卷会被创建。就像其名称所表示的那样，emptyDir 卷最初是空的。尽管 Pod 中的容器挂载 emptyDir 卷的路径可能相同也可能不同，但这些容器都可以读写 emptyDir 卷中相同的文件。当 Pod 因为某些原因被从节点上删除时，emptyDir 卷中的数据也会被永久删除。

容器崩溃并**不会**导致 Pod 被从节点上移除，因此容器崩溃期间 emptyDir 卷中的数据是安全的。

emptyDir 的一些用途：

- 缓存空间，例如基于磁盘的归并排序。
- 为耗时较长的计算任务提供检查点，以便任务能方便地从崩溃前状态恢复执行。
- 在 Web 服务器容器服务数据时，保存内容管理器容器获取的文件。

emptyDir.medium 字段用来控制 emptyDir 卷的存储位置。默认情况下，emptyDir 卷存储在该节点所使用的介质上；此处的介质可以是磁盘、SSD 或网络存储，这取决于你的环境。你可以将 emptyDir.medium 字段设置为 "Memory"，以告诉 Kubernetes 为你挂载 tmpfs（基于 RAM 的文件系统）。虽然 tmpfs 速度非常快，但是要注意它与磁盘不同，并且你所写入的所有文件都会计入容器的内存消耗，受容器内存限制约束。

你可以通过为默认介质指定大小限制，来限制 emptyDir 卷的存储容量。此存储是从节点临时存储中分配的。如果来自其他来源（如日志文件或镜像分层数据）的数据占满了存储，emptyDir 可能会在达到此限制之前发生存储容量不足的问题。

如果未指定大小，内存支持的卷将被设置为节点可分配内存的大小。

使用内存作为介质的 emptyDir 卷时，请查阅此处，了解有关资源管理方面的注意事项。

emptyDir 配置示例

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir:
      sizeLimit: 500Mi
```

emptyDir 内存配置示例

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir:
      sizeLimit: 500Mi
      medium: Memory
```

fc（光纤通道） {#fc}

fc 卷类型允许将现有的光纤通道块存储卷挂载到 Pod 中。可以使用卷配置中的参数 `targetWWNs` 来指定单个或多个目标 WWN（World Wide Names）。如果指定了多个 WWN，`targetWWNs` 期望这些 WWN 来自多路径连接。

你必须配置 FC SAN Zoning，以便预先向目标 WWN 分配和屏蔽这些 LUN（卷），这样 Kubernetes 主机才可以访问它们。

gcePersistentDisk（已弃用） {#gcepersistentdisk}

在 Kubernetes 中，所有针对树内 `gcePersistentDisk` 类型的操作都会被重定向到 `pd.csi.storage.gke.io` 驱动。

`gcePersistentDisk` 源代码树内卷存储驱动在 Kubernetes v1.17 版本中被弃用，在 v1.28 版本中被完全移除。

Kubernetes 项目建议你转为使用 Google Compute Engine Persistent Disk CSI 第三方存储驱动插件。

gitRepo（已禁用） {#gitrepo}

Kubernetes 不包含 `gitRepo` 卷驱动。提供使用此驱动方法的最后一个版本是 Kubernetes v1.35，并且自从 v1.11 小版本发布以来，它已被弃用。

如果需要制备已挂载 Git 仓库的 Pod，你可以将 EmptyDir 卷挂载到 Init 容器中，使用 Git 命令完成仓库的克隆操作，然后将 EmptyDir 卷挂载到 Pod 的容器中。

你可以使用 ValidatingAdmissionPolicy 这类策略来限制在你的集群中使用 gitRepo 卷。你可以使用以下通用表达语言（CEL）表达式作为策略的一部分，以拒绝使用 gitRepo 卷：

```
!has(object.spec.volumes) || !object.spec.volumes.exists(v, has(v.gitRepo))
```

hostPath {#hostpath}

hostPath 卷能将主机节点文件系统上的文件或目录挂载到你的 Pod 中。虽然这不是大多数 Pod 需要的，但是它为一些应用提供了强大的逃生舱。

使用 hostPath 类型的卷存在许多安全风险。如果可以，你应该尽量避免使用 hostPath 卷。例如，你可以改为定义并使用 local PersistentVolume。

如果你通过准入时的验证来限制对节点上特定目录的访问，这种限制只有在你额外要求所有 hostPath 卷的挂载都是**只读**的情况下才有效。如果你允许不受信任的 Pod 以读写方式挂载任意主机路径，则该 Pod 中的容器可能会破坏可读写主机挂载卷的安全性。

无论 hostPath 卷是以只读还是读写方式挂载，使用时都需要小心，这是因为：

- 访问主机文件系统可能会暴露特权系统凭证（例如 kubelet 的凭证）或特权 API（例如容器运行时套接字），这些可以被用于容器逃逸或攻击集群的其他部分。
- 具有相同配置的 Pod（例如基于 PodTemplate 创建的 Pod）可能会由于节点上的文件不同而在不同节点上表现出不同的行为。
- hostPath 卷的用量不会被视为临时存储用量。你需要自己监控磁盘使用情况，因为过多的 hostPath 磁盘使用量会导致节点上的磁盘压力。

hostPath 的一些用法有：

- 运行一个需要访问节点级系统组件的容器（例如一个将系统日志传输到集中位置的容器，使用只读挂载 /var/log 来访问这些日志）
- 让存储在主机系统上的配置文件可以被以只读方式访问；与普通 Pod 不同，静态 Pod 无法访问 ConfigMap。

hostPath 卷类型

除了必需的 path 属性外，你还可以选择为 hostPath 卷指定 type。

type 的可用值有：

- 取值：""；行为：空字符串（默认）用于向后兼容，这意味着在安装 hostPath 卷之前不会执行任何检查。
- 取值：DirectoryOrCreate；行为：如果在给定路径上什么都不存在，那么将根据需要创建空目录，权限设置为 0755，具有与 kubelet 相同的组和属主信息。
- 取值：Directory；行为：在给定路径上必须存在的目录。
- 取值：FileOrCreate；行为：如果在给定路径上什么都不存在，那么将在那里根据需要创建空文件，权限设置为 0644，具有与 kubelet 相同的组和所有权。
- 取值：File；行为：在给定路径上必须存在的文件。
- 取值：Socket；行为：在给定路径上必须存在的 UNIX 套接字。
- 取值：CharDevice；行为：**（仅 Linux 节点）** 在给定路径上必须存在的字符设备。
- 取值：BlockDevice；行为：**（仅 Linux 节点）** 在给定路径上必须存在的块设备。

FileOrCreate 模式**不会**创建文件的父目录。如果挂载文件的父目录不存在，Pod 将启动失败。为了确保这种模式正常工作，你可以尝试分别挂载目录和文件，如 hostPath 的 FileOrCreate 示例所示。

下层主机上创建的某些文件或目录只能由 root 用户访问。此时，你需要在特权容器中以 root 身份运行进程，或者修改主机上的文件权限，以便能够从 hostPath 卷读取数据（或将数据写入到 hostPath 卷）。

hostPath 配置示例

此清单将主机上的 `"/data/foo"` 目录挂载为 `hostpath-example-linux Pod` 中运行的单个容器内的 `"/foo"` 目录

容器中的挂载是只读的

apiVersion: v1 kind: Pod metadata: name: hostpath-example-linux spec: os: { name: linux } nodeSelector: kubernetes.io/os: linux containers:

- name: example-container image: registry.k8s.io/test-webserver volumeMounts:
 - mountPath: /foo name: example-volume readOnly: true
- name: example-volume

挂载 `"/data/foo"`，但仅当该目录已经存在时

hostPath: path: /data/foo # 主机上的目录位置 type: Directory # 此字段可选

此清单将主机上的 `"C:\Data\foo"` 目录挂载为 `hostpath-example-windows Pod`

中运行的单个容器内的 `"C:\foo"` 目录

容器中的挂载是只读的

apiVersion: v1 kind: Pod metadata: name: hostpath-example-windows spec: os: { name: windows } nodeSelector: kubernetes.io/os: windows containers:

- name: example-container image: microsoft/windowsservercore:1709
volumeMounts:
 - name: example-volume mountPath: "C:\foo" readOnly: true volumes:

从主机挂载 "C:\Data\foo"，但仅当该目录已经存在时

- name: example-volume hostPath: path: "C:\Data\foo" # 主机上的目录位置
type: Directory # 此字段可选

hostPath FileOrCreate 配置示例 {#hostpath-fileorcreate-example}

以下清单定义了一个 Pod，将 /var/local/aaa 挂载到 Pod 中的单个容器内。如果节点上还没有路径 /var/local/aaa，kubelet 会创建这一目录，然后将其挂载到 Pod 中。

如果 /var/local/aaa 已经存在但不是一个目录，Pod 会失败。此外，kubelet 还会尝试在该目录内创建一个名为 /var/local/aaa/1.txt 的文件（从主机的视角来看）；如果在该路径上已经存在某个东西且不是常规文件，则 Pod 会失败。

以下是清单示例：

```
apiVersion: v1
kind: Pod
metadata:
  name: test-webserver
spec:
  os: { name: linux }
  nodeSelector:
    kubernetes.io/os: linux
  containers:
  - name: test-webserver
    image: registry.k8s.io/test-webserver:latest
    volumeMounts:
    - mountPath: /var/local/aaa
      name: mydir
    - mountPath: /var/local/aaa/1.txt
      name: myfile
  volumes:
  - name: mydir
    hostPath:
      # 确保文件所在目录被创建成功。
      path: /var/local/aaa
```

```
    type: DirectoryOrCreate
- name: myfile
  hostPath:
    path: /var/local/aaa/1.txt
    type: FileOrCreate
```

image

image 卷源代表一个在 kubelet 主机上可用的 OCI 对象（容器镜像或工件）。

使用 image 卷源的一个例子是：

此卷在 Pod 启动时基于提供的 pullPolicy 值进行解析：

Always : kubelet 始终尝试拉取此引用。如果拉取失败，kubelet 会将 Pod 设置为 Failed。

Never : kubelet 从不拉取此引用，仅使用本地镜像或工件。如果本地没有任何镜像层存在，或者该镜像的清单未被缓存，则 Pod 会变为 Failed。

IfNotPresent : 如果引用在磁盘上不存在，kubelet 会进行拉取。如果引用不存在且拉取失败，则 Pod 会变为 Failed。

如果 Pod 被删除并重新创建，此卷会被重新解析，这意味着在 Pod 重新创建时将可以访问新的远程内容。在 Pod 启动期间解析或拉取镜像失败将导致容器无法启动，并可能显著增加延迟。如果失败，将使用正常的卷回退进行重试，并输出 Pod 失败的原因和相关消息。

此卷可以挂载的对象类型由主机上的容器运行时实现负责定义，至少必须包含容器镜像字段所支持的所有有效类型。OCI 对象将以只读方式被挂载到单个目录（spec.containers[*].volumeMounts[*].mountPath）中。

此外：

- 从 Kubernetes v1.33 开始，才支持容器的 subPath 或 subPathExpr 挂载（spec.containers[*].volumeMounts[*].subPath、spec.containers[*].volumeMounts[*].subPathExpr）。
- spec.securityContext.fsGroupChangePolicy 字段对这种卷没有效果。
- AlwaysPullImages 准入控制器也适用于此卷源，就像适用于容器镜像一样。

image 类型可用的字段如下：

reference : 要使用的工件引用。例如, 你可以指定 registry.k8s.io/conformance:v 来加载 Kubernetes 合规性测试镜像中的文件。其行为与 pod.spec.containers[*].image 相同。拉取 Secret 的组装方式与容器镜像所用的方式相同, 即通过查找节点凭据、服务账户镜像拉取 Secret 和 Pod 规约镜像拉取 Secret。此字段是可选的, 允许更高层次的配置管理在 Deployment 和 StatefulSet 这类工作负载控制器中默认使用或重载容器镜像。参阅容器镜像更多细节。

pullPolicy : 拉取 OCI 对象的策略。可能的值为: Always、Never 或 IfNotPresent。如果指定了 :latest 标记, 则默认为 Always, 否则默认为 IfNotPresent。

有关如何使用卷源的更多细节, 请参见 **Pod 使用镜像卷** 示例。

Pod 状态与 image 卷 {#image-volume-pod-status}

如果在你的集群中启用了 ImageVolumeWithDigest 特性门控, 那么每当你为 Pod 指定一个 image 卷时, kubelet 都会更新 Pod 状态, 记录作为卷来源使用的容器镜像的摘要 (digest) 。

下面是一个正在运行的 Pod 的简化示例, 以 YAML 表示, 其中已包含状态更新。请注意容器状态中 volumeMounts 下新增的 ImageRef 字段。

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: default
spec:
  containers:
  - name: shell
    command: ["sleep", "infinity"]
    image: docker.io/library/debian:12
    volumeMounts:
    - name: artifact
      mountPath: /data
  volumes:
  - name: artifact
    image:
      reference: quay.io/crio/artifact:v2
      pullPolicy: IfNotPresent
status:
  containerStatuses:
```

```
- containerID: containerd://examplecontainerid1234567890abcdef
  image: docker.io/library/debian:12
  imageID:
docker-pullable://docker.io/library/debian@sha256:3f1d6c17773a45c97bd8f158d665c9709
d7b29ed7917ac934086ad96f92e4510
  volumeMounts:
  - name: artifact
    mountPath: /data
    readOnly: true
  imageRef:
quay.io/crio/artifact@sha256:abcdef1234567890abcdef1234567890abcdef1234567890
ef1234567890
```

iscsi

iscsi 卷能将 iSCSI（基于 IP 的 SCSI）卷挂载到你的 Pod 中。不像 emptyDir 那样会在删除 Pod 的同时也会被删除，iscsi 卷的内容在删除 Pod 时会被保留，卷只是被卸载。这意味着 iscsi 卷可以被预先填充数据，并且这些数据可以在 Pod 之间共享。

在使用 iSCSI 卷之前，你必须拥有自己的 iSCSI 服务器，并在上面创建卷。

iSCSI 的一个特点是它可以同时被多个用户以只读方式挂载。这意味着你可以用数据集预先填充卷，然后根据需要在尽可能多的 Pod 上使用它。不幸的是，iSCSI 卷只能由单个使用者以读写模式挂载。不允许同时写入。

local

local 卷所代表的是某个被挂载的本地存储设备，例如磁盘、分区或者目录。

local 卷只能用作静态创建的持久卷。不支持动态配置。

与 hostPath 卷相比，local 卷能够以持久和可移植的方式使用，而无需手动将 Pod 调度到节点。系统通过查看 PersistentVolume 的节点亲和性配置，就能了解卷的节点约束。

然而，local 卷仍然取决于底层节点的可用性，并不适合所有应用程序。如果节点变得不健康，那么 local 卷也将变得不可被 Pod 访问。使用它的 Pod 将不能运行。使用 local 卷的应用程序必须能够容忍这种可用性的降低，以及因底层磁盘的耐用性特征而带来的潜在的数据丢失风险。

下面是一个使用 local 卷和 nodeAffinity 的持久卷示例：

```
apiVersion: v1
kind: PersistentVolume
```

```
metadata:
  name: example-pv
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd1
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
            - example-node
```

使用 local 卷时，你需要设置 PersistentVolume 对象的 nodeAffinity 字段。Kubernetes 调度器使用 PersistentVolume 的 nodeAffinity 信息来将使用 local 卷的 Pod 调度到正确的节点。

PersistentVolume 对象的 volumeMode 字段可被设置为 "Block"（而不是默认值 "Filesystem"），以将 local 卷作为原始块设备暴露出来。

使用 local 卷时，建议创建一个 StorageClass 并将其 volumeBindingMode 设置为 WaitForFirstConsumer。要了解更多信息，请参考 local StorageClass 示例。延迟卷绑定的操作可以确保 Kubernetes 在为 PersistentVolumeClaim 作出绑定决策时，会评估 Pod 可能具有的其他节点约束，例如：如节点资源需求、节点选择器、Pod 亲和性和 Pod 反亲和性。

你可以在 Kubernetes 之外单独运行静态驱动以改进对 local 卷的生命周期管理。请注意，此驱动尚不支持动态配置。有关如何运行外部 local 卷驱动，请参考 local 卷驱动用户指南。

如果不使用外部静态驱动来管理卷的生命周期，用户需要手动清理和删除 local 类型的持久卷。

nfs

nfs 卷能将 NFS（网络文件系统）挂载到你的 Pod 中。不像 emptyDir 那样会在删除 Pod 的同时也会被删除，nfs 卷的内容在删除 Pod 时会被保存，卷只是被卸载。这意味着 nfs 卷可以被预先填充数据，并且这些数据可以在 Pod 之间共享。

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /my-nfs-data
      name: test-volume
  volumes:
  - name: test-volume
    nfs:
      server: my-nfs-server.example.com
      path: /my-nfs-volume
      readOnly: true
```

在使用 NFS 卷之前，你必须运行自己的 NFS 服务器并将目标 share 导出备用。

还需要注意，不能在 Pod 规约中指定 NFS 挂载可选项。可以选择设置服务端的挂载可选项，或者使用 /etc/nfsmount.conf。此外，还可以通过允许设置挂载可选项的持久卷挂载 NFS 卷。

`persistentVolumeClaim` {#persistentvolumeclaim}

`persistentVolumeClaim` 卷用来将持久卷（`PersistentVolume`）挂载到 Pod 中。持久卷申领（`PersistentVolumeClaim`）是用户在不知道特定云环境细节的情况下“申领”持久存储（例如 iSCSI 卷）的一种方法。

更多详情请参考持久卷。

`portworxVolume`（已弃用） {#portworxvolume}

`portworxVolume` 是一个可伸缩的块存储层，能够以超融合（hyperconverged）的方式与 Kubernetes 一起运行。Portworx 支持对服务器上存储的指纹处理、基于存储能力进

行分层以及跨多个服务器整合存储容量。Portworx 可以以 in-guest 方式在虚拟机中运行，也可以在裸金属 Linux 节点上运行。

portworxVolume 类型的卷可以通过 Kubernetes 动态创建，也可以预先配备并在 Pod 内引用。下面是一个引用预先配备的 Portworx 卷的示例 Pod：

```
apiVersion: v1
kind: Pod
metadata:
  name: test-portworx-volume-pod
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /mnt
      name: pxvol
  volumes:
  - name: pxvol
    # 此 Portworx 卷必须已经存在
    portworxVolume:
      volumeID: "pxvol"
      fsType: ""
```

在 Pod 中使用 portworxVolume 之前，你要确保有一个名为 pxvol 的 PortworxVolume 存在。

Portworx CSI 迁移

在 Kubernetes 中，默认情况下，所有针对树内 Portworx 卷的操作都会被重定向到 pxd.portworx.com 容器存储接口（CSI）驱动。Portworx CSI 驱动程序必须安装在集群上。

投射 (projected) {#projected}

投射卷能将若干现有的卷来源映射到同一目录上。更多详情请参考投射卷。

secret

secret 卷用来给 Pod 传递敏感信息，例如密码。你可以将 Secret 存储在 Kubernetes API 服务器上，然后以文件的形式挂载到 Pod 中，无需直接与 Kubernetes 耦合。secret 卷

由 tmpfs（基于 RAM 的文件系统）提供存储，因此它们永远不会被写入非易失性（持久化的）存储器。

- 使用前你必须在 Kubernetes API 中创建 Secret。
- Secret 总是以 readOnly 的模式挂载。
- 容器以 subPath 卷挂载方式使用 Secret 时，将无法接收 Secret 的更新。

更多详情请参考配置 Secret。

使用 subPath {#using-subpath}

有时，在单个 Pod 中共享卷以供多方使用是很有用的。volumeMounts[*].subPath 属性可用于指定所引用的卷内的子路径，而不是其根路径。

下面例子展示了如何配置某包含 LAMP 堆栈（Linux、Apache、MySQL、PHP）的 Pod 使用同一共享卷。此示例中的 subPath 配置不建议在生产环境中使用。PHP 应用的代码和相关数据映射到卷的 html 文件夹，MySQL 数据库存储在卷的 mysql 文件夹中：

```
apiVersion: v1
kind: Pod
metadata:
  name: my-lamp-site
spec:
  containers:
    - name: mysql
      image: mysql
      env:
        - name: MYSQL_ROOT_PASSWORD
          value: "rootpasswd"
      volumeMounts:
        - mountPath: /var/lib/mysql
          name: site-data
          subPath: mysql
    - name: php
      image: php:7.0-apache
      volumeMounts:
        - mountPath: /var/www/html
          name: site-data
          subPath: html
  volumes:
    - name: site-data
```

```
persistentVolumeClaim:  
  claimName: my-lamp-site-data
```

使用带有扩展环境变量的 subPath {#using-subpath-expanded-environment}

使用 subPathExpr 字段可以基于 downward API 环境变量来构造 subPath 目录名。
subPath 和 subPathExpr 属性是互斥的。

在这个示例中，Pod 使用 subPathExpr 来 hostPath 卷 /var/log/pods 中创建目录 pod1。
hostPath 卷采用来自 downwardAPI 的 Pod 名称生成目录名。宿主机目录
/var/log/pods/pod1 被挂载到容器的 /logs 中。

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: pod1  
spec:  
  containers:  
    - name: container1  
      env:  
        - name: POD_NAME  
          valueFrom:  
            fieldRef:  
              apiVersion: v1  
              fieldPath: metadata.name  
      image: busybox:1.28  
      command: [ "sh", "-c", "while [ true ]; do echo 'Hello'; sleep 10; done | tee -a  
/logs/hello.txt" ]  
      volumeMounts:  
        - name: workdir1  
          mountPath: /logs  
            # 包裹变量名的是小括号，而不是大括号  
          subPathExpr: $(POD_NAME)  
      restartPolicy: Never  
  volumes:  
    - name: workdir1  
      hostPath:  
        path: /var/log/pods
```

资源 {#resources}

emptyDir 卷的存储介质（例如磁盘、SSD 等）是由保存 kubelet 数据的根目录（通常是 /var/lib/kubelet）的文件系统的介质确定。Kubernetes 对 emptyDir 卷或者 hostPath 卷可以消耗的空间没有限制，容器之间或 Pod 之间也没有隔离。

要了解如何使用资源规约来请求空间，可参考[如何管理资源](#)。

树外（Out-of-Tree）卷插件 {#out-of-tree-volume-plugins}

Out-of-Tree 卷插件包括和 FlexVolume（已弃用）。它们使存储供应商能够创建自定义存储插件，而无需将插件源码添加到 Kubernetes 代码仓库。

以前，所有卷插件（如上面列出的卷类型）都是“树内（In-Tree）”的。“树内”插件是与 Kubernetes 的核心组件一同构建、链接、编译和交付的。这意味着向 Kubernetes 添加新的存储系统（卷插件）需要将代码合并到 Kubernetes 核心代码库中。

CSI 和 FlexVolume 都允许独立于 Kubernetes 代码库开发卷插件，并作为扩展部署（安装）在 Kubernetes 集群上。

对于希望创建树外（Out-Of-Tree）卷插件的存储供应商，请参考[卷插件常见问题](#)。

CSI

容器存储接口（CSI）为容器编排系统（如 Kubernetes）定义标准接口，以将任意存储系统暴露给它们的容器工作负载。

更多详情请阅读[CSI 设计方案](#)。

Kubernetes v1.13 废弃了对 CSI 规范版本 0.2 和 0.3 的支持，并将在以后的版本中删除。

CSI 驱动可能并非兼容所有的 Kubernetes 版本。请查看特定 CSI 驱动的文档，以了解各个 Kubernetes 版本所支持的部署步骤以及兼容性列表。

一旦在 Kubernetes 集群上部署了 CSI 兼容卷驱动程序，用户就可以使用 csi 卷类型来挂接、挂载 CSI 驱动所提供的卷。

csi 卷可以在 Pod 中以三种方式使用：

- 通过 PersistentVolumeClaim 对象引用
- 使用一般性的临时卷
- 使用 CSI 临时卷，前提是驱动支持这种用法

存储管理员可以使用以下字段来配置 CSI 持久卷：

- **driver**: 指定要使用的卷驱动名称的字符串值。这个值必须与 CSI 驱动程序在 `GetPluginInfoResponse` 中返回的值相对应；该接口定义在 CSI 规范中。Kubernetes 使用所给的值来标识要调用的 CSI 驱动程序；CSI 驱动程序也使用该值来辨识哪些 PV 对象属于该 CSI 驱动程序。
- **volumeHandle**: 唯一标识卷的字符串值。该值必须与 CSI 驱动在 `CreateVolumeResponse` 的 `volume_id` 字段中返回的值相对应；接口定义在 CSI 规范中。在所有对 CSI 卷驱动程序的调用中，引用该 CSI 卷时都使用此值作为 `volume_id` 参数。
- **readOnly**: 一个可选的布尔值，指示通过 `ControllerPublished` 关联该卷时是否设置该卷为只读。默认值是 `false`。该值通过 `ControllerPublishVolumeRequest` 中的 `readonly` 字段传递给 CSI 驱动。
- **fsType**: 如果 PV 的 `VolumeMode` 为 `Filesystem`，那么此字段指定挂载卷时应该使用的文件系统。如果卷尚未格式化，并且支持格式化，此值将用于格式化卷。此值可以通过 `ControllerPublishVolumeRequest`、`NodeStageVolumeRequest` 和 `NodePublishVolumeRequest` 的 `VolumeCapability` 字段传递给 CSI 驱动。
- **volumeAttributes**: 一个字符串到字符串的映射表，用来设置卷的静态属性。该映射必须与 CSI 驱动程序返回的 `CreateVolumeResponse` 中的 `volume.attributes` 字段的映射相对应；CSI 规范中有相应的定义。该映射通过 `ControllerPublishVolumeRequest`、`NodeStageVolumeRequest` 和 `NodePublishVolumeRequest` 中的 `volume_context` 字段传递给 CSI 驱动。
- **controllerPublishSecretRef**: 对包含敏感信息的 Secret 对象的引用；该敏感信息会被传递给 CSI 驱动来完成 `CSI ControllerPublishVolume` 和 `ControllerUnpublishVolume` 调用。此字段是可选的；在不需要 Secret 时可以是空的。如果 Secret 包含多个 Secret 条目，则所有的 Secret 条目都会被传递。
- **nodeExpandSecretRef**: 对包含敏感信息的 Secret 对象的引用，该信息会传递给 CSI 驱动以完成 `CSI NodeExpandVolume` 调用。此字段是可选的，如果不需要 Secret，则可能是空的。如果 Secret 包含多个 Secret 条目，则传递所有 Secret 条目。当你为节点初始化的卷扩展配置 Secret 数据时，kubelet 会通过 `NodeExpandVolume()` 调用将该数据传递给 CSI 驱动。所有受支持的 Kubernetes 版本都提供 `nodeExpandSecretRef` 字段，并且默认可用。Kubernetes v1.25 之前的版本不包括此支持。

为每个 kube-apiserver 和每个节点上的 kubelet 启用名为 `CSINodeExpandSecret` 的特性门控。自 Kubernetes 1.27 版本起，此特性已默认启用，无需显式启用特

性门控。在节点初始化的存储大小调整操作期间，你还必须使用支持或需要 Secret 数据的 CSI 驱动。

- `nodePublishSecretRef`：对包含敏感信息的 Secret 对象的引用。该信息传递给 CSI 驱动来完成 CSI `NodePublishVolume` 调用。此字段是可选的，如果不需要 Secret，则可能是空的。如果 Secret 对象包含多个 Secret 条目，则传递所有 Secret 条目。
- `nodeStageSecretRef`：对包含敏感信息的 Secret 对象的引用，该信息会传递给 CSI 驱动以完成 CSI `NodeStageVolume` 调用。此字段是可选的，如果不需要 Secret，则可能是空的。如果 Secret 包含多个 Secret 条目，则传递所有 Secret 条目。

CSI 原始块卷支持 {#csi-raw-block-volume-support}

具有外部 CSI 驱动程序的供应商能够在 Kubernetes 工作负载中实现原始块卷支持。

你可以和以前一样，安装自己的带有原始块卷支持的 PV/PVC，采用 CSI 对此过程没有影响。

CSI 临时卷 {#csi-ephemeral-volumes}

你可以直接在 Pod 规约中配置 CSI 卷。采用这种方式配置的卷都是临时卷，无法在 Pod 重新启动后继续存在。进一步的信息可参阅临时卷。

有关如何开发 CSI 驱动的更多信息，请参考 `kubernetes-csi` 文档。

Windows CSI 代理 {#windows-csi-proxy}

CSI 节点插件需要执行多种特权操作，例如扫描磁盘设备和挂载文件系统等。这些操作在每个宿主机操作系统上都是不同的。对于 Linux 工作节点而言，容器化的 CSI 节点插件通常部署为特权容器。对于 Windows 工作节点而言，容器化 CSI 节点插件的特权操作是通过 `csi-proxy` 来支持的。`csi-proxy` 是一个由社区管理的、独立的可执行二进制文件，需要被预安装到每个 Windows 节点上。

要了解更多的细节，可以参考你要部署的 CSI 插件的部署指南。

从树内插件迁移到 CSI 驱动程序 {#migrating-to-csi-drivers-from-in-tree-plugins}

`CSIMigration` 特性针对现有树内插件的操作会被定向到相应的 CSI 插件（应已安装和配置）。因此，操作员在过渡到取代树内插件的 CSI 驱动时，无需对现有存储类、PV 或 PVC（指树内插件）进行任何配置更改。

即使你针对这种卷完成了 CSI 迁移且你升级到不再内置对这种存储类别的支持的 Kubernetes 版本，现有的由树内卷插件所创建的 PV 在未来无需进行任何配置更改就可以使用，

作为迁移的一部分，你或其他集群管理员**必须**安装和配置适用于该存储的 CSI 驱动。Kubernetes 不会为你安装该软件。

在完成迁移之后，你也可以定义新的 PVC 和 PV，引用原来的、内置的集成存储。只要你安装并配置了适当的 CSI 驱动，即使是全新的卷，PV 的创建仍然可以继续工作。实际的存储管理现在通过 CSI 驱动来进行。

所支持的操作和特性包括：配备（Provisioning）/删除、挂接（Attach）/解挂（Detach）、挂载（Mount）/卸载（Unmount）和调整卷大小。

上面的卷类型节列出了支持 CSIMigration 并已实现相应 CSI 驱动程序的树内插件。

flexVolume（已弃用）{#flexvolume}

FlexVolume 是一个使用基于 exec 的模型来与驱动程序对接的树外插件接口。用户必须在每个节点上的预定义卷插件路径中安装 FlexVolume 驱动程序可执行文件，在某些情况下，控制平面节点中也要安装。

Pod 通过 flexvolume 树内插件与 FlexVolume 驱动程序交互。

下面的 FlexVolume 插件以 PowerShell 脚本的形式部署在宿主机系统上，支持 Windows 节点：

- SMB
- iSCSI

FlexVolume 已被弃用。推荐使用树外 CSI 驱动来将外部存储整合进 Kubernetes。

FlexVolume 驱动的维护者应开发一个 CSI 驱动并帮助用户从 FlexVolume 驱动迁移到 CSI。FlexVolume 用户应迁移工作负载以使用对等的 CSI 驱动。

挂载卷的传播 {#mount-propagation}

挂载卷的传播是一项底层功能，不能在所有类型的卷中以一致的方式工作。建议只在 hostPath 或基于内存的 emptyDir 卷中使用。详情请参考讨论。

挂载卷的传播能力允许将容器安装的卷共享到同一 Pod 中的其他容器，甚至共享到同一节点上的其他 Pod。

卷的挂载传播特性由 `containers[*].volumeMounts` 中的 `mountPropagation` 字段控制，它的值包括：

- `None` - 此卷挂载将不会感知到主机后续在此卷或其任何子目录上执行的挂载变化。类似的，容器所创建的卷挂载在主机上是不可见的。这是默认模式。

该模式等同于 `mount(8)` 中描述的 `rprivate` 挂载传播选项。

然而，当 `rprivate` 传播选项不适用时，CRI 运行时可以转为选择 `rslave` 挂载传播选项（即 `HostToContainer`）。当挂载源包含 Docker 守护进程的根目录（`/var/lib/docker`）时，`cri-dockerd`（Docker）已知可以选择 `rslave` 挂载传播选项。

- `HostToContainer` - 此卷挂载将会感知到主机后续针对此卷或其任何子目录的挂载操作。

换句话说，如果主机在此挂载卷中挂载任何内容，容器将能看到它被挂载在那里。

类似的，配置了 `Bidirectional` 挂载传播选项的 Pod 如果在同一卷上挂载了内容，挂载传播设置为 `HostToContainer` 的容器都将能看到这一变化。

该模式等同于 `mount(8)` 中描述的 `rslave` 挂载传播选项。

- `Bidirectional` - 这种卷挂载和 `HostToContainer` 挂载表现相同。另外，容器创建的卷挂载将被传播回至主机和使用同一卷的所有 Pod 的所有容器。

该模式的典型用例是带有 `FlexVolume` 或 `CSI` 驱动的 Pod，或者需要通过 `hostPath` 卷在主机上挂载某些东西的 Pod。

该模式等同于 `mount(8)` 中描述的 `rshared` 挂载传播选项。

`Bidirectional` 形式的挂载传播可能比较危险。它可以破坏主机操作系统，因此它只被允许在特权容器中使用。强烈建议你熟悉 Linux 内核行为。此外，由 Pod 中的容器创建的任何卷挂载必须在终止时由容器销毁（卸载）。

只读挂载 {#read-only-mounts}

通过将 `.spec.containers[*].volumeMounts[*].readOnly` 字段设置为 `true` 可以使挂载只读。这不会使卷本身只读，但该容器将无法写入此卷。Pod 中的其他容器可以以读写方式挂载同一个卷。

在 Linux 上，只读挂载默认不会以递归方式只读。假如有一个 Pod 将主机的 /mnt 挂载为 hostPath 卷。如果在 /mnt/ 上有另一个以读写方式挂载的文件系统（如 tmpfs、NFS 或 USB 存储），即使挂载本身被指定为只读，挂载到容器中的卷 /mnt/ 也是可写的。

递归只读挂载 {#recursive-read-only-mounts}

通过为 kubelet 和 kube-apiserver 设置 RecursiveReadOnlyMounts 特性门控，并为 Pod 设置 .spec.containers[*].volumeMounts[*].recursiveReadOnly 字段，递归只读挂载可以被启用。

允许的值为：

- Disabled（默认）：无效果。
- Enabled：使挂载递归只读。需要满足以下所有要求：
 - readOnly 设置为 true
 - mountPropagation 不设置，或设置为 None
 - 主机运行 Linux 内核 v5.12 或更高版本
 - CRI 级别的容器运行时支持递归只读挂载
 - OCI 级别的容器运行时支持递归只读挂载

如果其中任何一个不满足，递归只读挂载将会失败。

- IfPossible：尝试应用 Enabled，如果内核或运行时类不支持该特性，则回退为 Disabled。

示例：

当此属性被 kubelet 和 kube-apiserver 识别到时，.status.containerStatuses[*].volumeMounts[*].recursiveReadOnly 字段将被设置为 Enabled 或 Disabled。

实现 {#implementations-rro}

以下容器运行时已知支持递归只读挂载。

CRI 级别：

- containerd，自 v2.0 起
- CRI-O，自 v1.30 起

OCI 级别：

- runc, 自 v1.1 起
- crun, 自 v1.8.6 起

参考使用持久卷部署 WordPress 和 MySQL 示例。